FRED TALKS

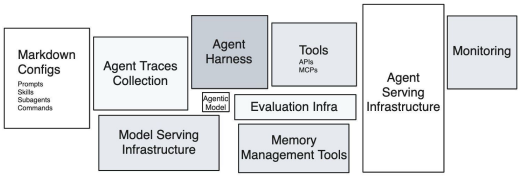
22nd August 2026

CONTENTS

Hidden Technical Debt of AI Systems: Agent Runtime

HAN, NOT SOLO · 27 APR 2026 · SOURCE

Eleven years ago, Sculley et al. drew the diagram everyone in MLOps has seen: a tiny black box labeled “ML Code” surrounded by a sprawl of much larger boxes — data collection, feature extraction, configuration, monitoring, serving infrastructure. The point of the diagram was that the model code is the smallest piece of a real ML system, and that everything else is where the technical debt accumulates.



Hidden Technical Debt in Agentic AI Systems

The same diagram is being redrawn for agents. The agentic model call is the small box. The largest box to the right that is currently driving most of the spend and influencing how system architecture will be done in the futureincidents, is the **agent runtime**, or agent serving infrastructure.

The agent is not the model. The agent is the harness plus the model, running inside the runtime. And almost nobody is treating it that way.

What an Agent Runtime Actually Is

A agentic model by itself maps from text input to text output. Other modalities are in play, but the primary fabric is still text. An agent is what you get when you wrap the agentic model with harnesses so it can take actions, observe effects, feed those observation back into the next call. The execution environment where that happens is the runtime.

Concretely, an agent runtime is the union of:

- **compute substrate** — a container, microVM, or full VM where code runs.
- **filesystem** the agent can read and write, often with snapshot and rollback semantics.

- **tools** — shell, code interpreter, browser, file editor, MCP servers — exposed to the model as callable interfaces.
- **network boundary** that defines what the agent can reach and what can reach it.
- **state model** that decides what persists across turns, across episodes, and across users.
- **lifecycle controller** that starts, suspends, snapshots, resumes, and tears down environments.

In the taxonomy of RL environments for LLM agents, these are the (harness) and (state) components. In production, this is the part that decides whether your agent finishes a task in eight seconds or eight minutes, whether two users can share an environment safely, and whether a malicious prompt can read your secrets.

Most teams shipping agents today have not built this layer. They have rented it, glued it together from cloud primitives that were designed for a different workload, or simply not thought about it yet. That last group is the one accumulating the most debt.

Why Sandboxing Is Not Optional

Models hallucinate code. They run `rm -rf`. They paste credentials into curl commands. They follow instructions embedded in untrusted documents. They retry the same failing command in a tight loop and burn through a quota. None of these are exotic edge cases — they are the modal behavior of capable models when handed unrestricted tool access.

The sandbox is what stops these behaviors from becoming incidents. Four reasons it has to exist as a first-class layer rather than a hopeful disclaimer in the system prompt:

Isolation against the model's mistakes. A coding agent that mounts your repo and has shell access can, and eventually will, delete the wrong directory. Filesystem isolation, copy-on-write snapshots, and per-session ephemerality turn destructive actions into recoverable ones.

Isolation against prompt injection. The agent reads tool outputs. Tool outputs are attacker-controlled the moment the agent visits a webpage, opens a PDF, or processes a customer email. A sandbox is the only thing standing between an injected instruction and your production database. This is the agent-systems version of the SQL injection lesson, and we are at roughly the 2003 stage of learning it.

Multi-tenancy at training scale. RL training spins up thousands of concurrent rollouts. Each rollout needs its own filesystem, its own process tree, its own network namespace. Without strong isolation primitives, one rollout's flaky shell command takes down a neighbor's training step.

Reproducibility. A sandbox you can snapshot is a sandbox you can replay. Replay is how you debug a six-hour agent trajectory without re-running the whole thing. It is also how you turn a production failure into a regression test.

A web app's runtime can be sloppy because the user is the one driving and a refresh fixes most things. An agent's runtime cannot, because the agent will keep going long after a human would have stopped to ask a question.

The Cognition team is direct about this in their post-mortem on building Devin's cloud agent infrastructure: containerized agents share a kernel, and a single compromised session can reach every other container's filesystem, credentials, and network connections. Because agents generate their own code, run arbitrary commands, and probe the environment in unpredictable ways, kernel-escape is not a theoretical risk — it is the working assumption. Their conclusion, after more than a year of hypervisor engineering, is the same one the broader infrastructure community has converged on for any untrusted-code workload: VM-level isolation, where each session gets its own kernel and there is no shared attack surface.

Manus team has famously built and snapped shotted all their agent runtimes in a restorable format so users can come back in the future to replay what agents have done.

The Isolation Primitive Stack

Most of the differentiation between sandbox vendors lives one layer down, in the isolation primitive they pick. There are five primitives in serious use, with very different trade-offs.

Primitive	Isolation model	Cold start	Workload fit	Notable users
Linux containers (runc, Podman)	Shared host kernel, namespaces + cgroups + seccomp	~100ms	Trusted code, internal CI	Docker, Kubernetes default
Firecracker	KVM-based microVM, dedicated kernel per VM, ~5MB Rust VMM	~125ms boot, sub-second from snapshot	Untrusted code at high density	AWS Lambda, Fargate, E2B, Fly.io

gVisor	Userspace kernel intercepting syscalls; runc-compatible runtime	Container-class	Defense in depth where a microVM is overkill; GPU virtualization	Google Cloud Run, App Engine, Modal
Kata Containers	Lightweight VM per pod, OCI-compatible	Few hundred ms	Multi-tenant Kubernetes	Confidential Containers, some managed K8s
V8 isolates	Per-tenant JS heap inside a single process	Sub-millisecond	JavaScript-only, no arbitrary binaries	Cloudflare Workers, Deno Deploy

A few things are worth saying out loud about this table.

Containers are not a sandbox for agent code. They are a packaging and resource-control mechanism. The shared kernel means a kernel exploit, a misconfigured capability, or a sloppy seccomp profile takes down the isolation boundary. It is not okay for an agent that may execute attacker-controlled instructions hidden in a web page.

Firecracker is the de facto industry primitive for this category. AWS open-sourced it in 2018 to power Lambda and Fargate, and almost all agent-sandbox startup runs on top of it, including E2B, Fly.io, Vercel Sandbox. It boots a stripped-down Linux kernel inside KVM in roughly 125 milliseconds and uses about five megabytes of memory for the VMM itself. The combination of strong isolation, fast boot, high density is what makes thousands of concurrent rollouts economically viable.

gVisor is the middle path. It runs a userspace kernel that intercepts and re-implements Linux syscalls, giving you stronger isolation than namespaces without paying the full cost of a hardware VM. The trade-off is that some syscalls are slower or unsupported, which matters for workloads that hit the kernel hard. Google uses it for Cloud Run and App Engine; it is the right pick when you want defense-in-depth on top of containers but cannot move to microVMs.

Kata Containers fit a specific niche. They are an OCI-compatible runtime that wraps each pod in a lightweight VM, which lets you run untrusted workloads on Kubernetes without rewriting the orchestration layer. The cost is that you inherit Kubernetes' assumptions about pod lifetime, which do not match how agents actually behave.

V8 isolates are the wrong primitive for general agent workloads. They are extraordinary for JavaScript-only edge compute but an agent that needs to run arbitrary Python, install packages, drive a browser, or execute compiled binaries cannot live inside a V8 heap. Cloudflare's own Sandbox product addresses this by adding container-class compute alongside isolates.

The picks higher up the stack inherit these trade-offs. When a sandbox vendor advertises “VM-level isolation” they almost always mean Firecracker. When they advertise “millisecond cold starts” without VM isolation, they usually mean V8 isolates with a different threat model. The vendor brochure abstracts the primitive away; the threat model and the workload do not.

The Startup Sandbox Landscape

A category of “sandbox-as-a-service” companies has emerged in the last two years, almost all of them building on top of the primitives above. Northflank’s roundup of Modal Sandboxes alternatives is a good market snapshot — the differentiation is in the developer ergonomics, the snapshot model, and the mix of tools that come pre-wired, not in the isolation layer underneath.

Vendor	Isolation	Snapshot model	Cold start	Notable design choice
Modal	gVisor	Filesystem diffs from base image	Sub-second from snapshot	GPU support
E2B	Firecracker microVM	Full VM snapshot	~150ms	Open-source SDK, language-agnostic, popular in the agent dev community
Daytona	Containers / VMs	Forkable workspaces	Few seconds	Forked dev environments, OCI-compatible
Northflank	Containers, GPU-aware	Persistent volumes	Container-class	GPU sandboxes for agents that train or run inference inside the loop
Browserbase / Steel / Hyperbrowser	Containerized browsers	Session replay	Seconds	Browser-only runtimes for web agents — DOM, screenshots, CDP
Cloudflare Workers Sandbox	V8 isolates + containers	Object snapshots	Milliseconds	Edge-first, shared-nothing model
Vercel Sandbox	Firecracker	Snapshot from build	Sub-second	Tied to Vercel’s deploy and preview model

The reference Modal case study is Ramp Inspect: a background coding agent that now writes more than half of Ramp’s merged pull requests, with each session running in its own Modal sandbox containing Postgres, Redis, Temporal, RabbitMQ, a VS Code server, and a VNC stack

with Chromium. Filesystem snapshots are refreshed every thirty minutes by a cron, so a new session is working on a prompt within seconds. The lesson buried in that architecture is that the agent's productivity is bounded by the runtime's startup time, not by the model's tokens-per-second.

The browser-runtime category exists because driving Chromium is its own engineering problem — CDP, profiles, residential IPs, captcha handling, session persistence — and no general-purpose sandbox has it built in. Expect this to consolidate into the general-purpose vendors over the next eighteen months, the same way headless browsers absorbed into the major test frameworks.

It is also worth noting what nobody on this list does themselves. None of these vendors writes their own hypervisor. They all stand on top of the primitives in the previous section, which is what lets the category exist. The startups that have tried to build a complete stack — including the hypervisor — have ended up looking more like Cognition: years of engineering investment to get something that is finally indistinguishable from "we run on Firecracker with custom snapshot logic."

The Hyperscaler Sandbox Landscape

The hyperscalers got to this market late and are still catching up.

AWS Bedrock AgentCore ships a Code Interpreter and a Browser Tool as managed runtimes for agents. The isolation is microVM-class, the integrations point at the rest of the AWS data plane, and the pricing model is per-session. However, it has major gaps relative to the startups, where the runtime has to be coded and shipped like a Lambda function, instead of having the flexibility to support heterogeneous workloads, e.g, different agent harness + model combinations. Not a major problem for production, but a major problem for research and development.

Azure Container Apps Dynamic Sessions uses Hyper-V isolation and advertises sub-second start times for code interpreter sessions. It is the most directly comparable hyperscaler offering to E2B and Modal in terms of intent. The integration story is strongest if you are already on Azure OpenAI and AKS. It also suffers from the flexibility to support heterogeneous workloads.

GCP Cloud Run Sandboxes is ahead of the pack with the usability similar to Modal, E2B, Daytona and the likes. It's built on top of Cloud Run, which uses gVisor. Supports heterogeneous workloads.

Lambda, Fargate, App Runner, Cloud Run. It is tempting to use these as agent sandboxes because they are cheap and already in your account. They are designed for stateless, request-response workloads with strict execution-time limits and no native snapshot model. They will work for a demo. They will not work for an agent that runs for six hours, mutates a filesystem, and needs to fork mid-trajectory.

The hyperscaler offerings are converging on the right shape — microVM isolation, fast snapshots, managed tools — but the primitives underneath were built for web workloads. The startups had the advantage of building on top of those primitives without inheriting the legacy product surface area.

Experimentation and Production Want Different Runtimes

One of the major difference how AI engineering differes from traditional web development is the difference in development and production infrastructure requiremetns. Here's the difference between your training cluster (if you are training models), experimentation and evaluation runs, vs production workload.

Dimension	Experimentation / Training	Production / Serving
Concurrency shape	Thousands of parallel rollouts, bursty	One session per user, steady-state
Cold start tolerance	Critical — 5s × 10k rollouts is real money	Forgivable — users wait
State model	Fork, branch, replay, snapshot	Durable, per-user, auditable
Network policy	Often offline or recorded for determinism	Open internet, real APIs
Failure model	Drop the rollout, sample more	Retry, degrade, page someone
Observability	Full trace, every token, replayable	SLOs, error budgets, sampling
Cost model	Spot, preemptible, batch	Reserved, predictable, latency-bound
Determinism	Required for reproducible runs	Often counterproductive
Lifetime	Seconds to minutes	Minutes to hours, or pinned for a session

A training rollout wants to start in 200 milliseconds, run for thirty seconds against a frozen snapshot of the world, get scored, and disappear. A production session wants to start in two seconds, hold state for a forty-minute conversation, talk to the live internet, and survive a transient failure without losing the user's work.

The production side has a requirement that almost nobody surfaces in eval reports: the agent needs to survive the **async gaps** in real engineering work. Cognition's experience report is the cleanest articulation of this — an agent opens a PR, waits on CI, responds to a review comment, reruns tests, pushes a follow-up commit. Between each step there are minutes, hours, sometimes days where the agent's working state has to persist. A containerized agent can only survive these gaps by burning compute to stay alive, and if the container is rescheduled or times out the session is lost. Their solution was hypervisor-level snapshotting of the full machine state — memory, process tree, filesystem — so compute shuts down while the agent is idle and resumes exactly where it left off when a CI result arrives. Building that took longer than any other piece of infrastructure they had shipped to date.

Optimizing the same runtime for both training and production is how you end up with a system that is too slow for training and too brittle for production. Most teams discover this after the first time they try to "just deploy what we trained on" and find the latency budget eaten by a startup model that was fine when amortized over ten thousand rollouts and intolerable when paid by a single user staring at a spinner.

Dev/Prod Parity Is the Real Problem

In 2011, Twelve-Factor App told us to keep development, staging, and production as similar as possible. That advice was about reducing the gap between where the engineer types code and where the code runs. For agents, the gap that matters is between where the agent **trained** and **experimented** and where the agent **runs**.

An agent learns the runtime. Tool latencies, failure modes, shell quirks, filesystem layout, the exact way `ls` formats output, the way the browser resolves redirects. The model picks up all of it during training and bakes it into the policy, or the optimizer (RLM or GEPA) picks up all of it during training and bakes it into the harness or the prompts. Move these to a different runtime and the behavior shifts. Tools that were idempotent become flaky. Commands that were instant now block. Snapshots that existed disappear. The agent has no abstract concept of "shell"; it has a concept of "the shell I trained on."

This is a new flavor of distributional shift. It is not the data shift the MLOps community has been worrying about for a decade. It is **runtime shift**, and it shows up as silent quality regressions that no eval catches because the eval runs in the training runtime.

There are three honest paths through this:

Co-locate train and prod on the same runtime. Pick one sandbox provider, run RL rollouts on it, run production sessions on it, accept the lock-in. This is what the most disciplined AI engineering teams are doing.

Define a runtime contract and enforce it on both sides. A small, versioned interface — "this is the shell, these are the tools, these are the latencies, these are the failure modes" — that you implement once on your training infrastructure and once on your production infrastructure. The contract is the abstraction; the sandbox underneath can vary. This is harder than it sounds because the contract has to cover not just the API surface but the timing and failure semantics that the agent has learned.

Train against production noise. Inject latency, errors, and tool failures during training so the agent's policy is robust to runtime variance instead of dependent on a specific runtime. Step-DeepResearch reports tangible gains from injecting 5–10% tool errors during training. This is the runtime analog of dropout.

The wrong answer, the one most teams are unconsciously picking, is to pick a sandbox for production as a software engineering solution, and forget all about the requirements from AI and machine learning, then spend the next quarters chasing flakiness of agent performances.

The Bill That Is Coming Due

Sculley's argument was that ML systems accumulate technical debt in places that traditional software engineering does not have language for — feedback loops, configuration sprawl, undeclared consumers, glue code. Agent systems inherit all of that and add a new line item: the runtime debt.

Runtime debt looks like this. A team picks the sandbox that was easiest to integrate at prototype time. Production traffic exposes a different mix of failure modes. The team patches around the gap with retries, longer timeouts, and prompt-engineered apologies. The agent's behavior gets entangled with that runtime's quirks with whack-a-mole without disciplined research and development. A year later, switching runtimes is a six-month migration because nobody can predict which behaviors will move.

The agent has to live on a runtime. The teams that internalize that early will spend the next two years compounding the advantage. The teams that do not will spend the next two years paying down a debt they did not know they were taking on.

References

- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. NeurIPS. https://papers.nips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html

- Lee, H. (2026). A Taxonomy of RL Environments for LLM Agents. Han, Not Solo. https://leehanchung.github.io/blogs/2026/03/21/rl-environments-for-llm-agents/

- Workman, G. (2026). How Ramp built a full-context background coding agent on Modal. Modal Blog. https://modal.com/blog/how-ramp-built-a-full-context-background-coding-agent-on-modal

- Lopopolo, R. (2026). Harness engineering: leveraging Codex in an agent-first world. OpenAI. https://openai.com/index/harness-engineering/

- Microsoft. (2024). Azure Container Apps Dynamic Sessions. https://learn.microsoft.com/en-us/azure/container-apps/sessions

- Firecracker. AWS open-source microVM. https://firecracker-microvm.github.io/

- Northflank. (2026). Top Modal Sandboxes Alternatives for Secure AI Code Execution. https://northflank.com/blog/top-modal-sandboxes-alternatives-for-secure-ai-code-execution


```
@article{
    leehanchung,
    author = {Lee, Hanchung},
    title = {Hidden Technical Debt of AI Systems: Agent Runtime},
    year = {2026},
    month = {04},
    day = {24},
    howpublished = {\url{https://leehanchung.github.io}},
    url = {https://leehanchung.github.io/blogs/2026/04/24/hidden-tech
}
```

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

The lies I used to tell myself

CATE HALL · 23 JAN 2026 · SOURCE

1. What doesn't kill you makes you stronger.

When I was 31, my husband of four years, who I loved more than my own life, left me completely out of the blue. We'd had an incredible, epic romance — drawn together like magnets, we married on our third date and spent the years that followed burrowing ever closer together. Maybe that was the problem — I can't say for sure, because I never got a satisfying explanation out of him. One day I was the protagonist of a Princess Bride-worthy love story; 48 hours later, he moved out.

From the time we met, I had never contemplated the possibility of living without him, and I didn't want to. I wanted to die for a long time — at least a couple of years. But as I slowly learned to sequester away thoughts of him and the emotions they brought up, I became convinced of a silver lining: The experience would make me impenetrable. If I survived it, I would know I could survive anything, and nothing would be able to hurt me again.

It took the better part of a decade and another deeply identity-shifting relationship, with the man who became my second husband, for me to see that I had it all wrong: Impenetrability means avoiding the parts of life that are most life-like. There's a reason the visual metaphor for love is an arrow.

The near-fatal catastrophes of our lives don't always make us stronger. Surviving one hard thing does provide evidence that you can survive other hard things, which is easy to *mistake* for getting stronger. But in reality they often just make us brittle, make us dissociate and flee from the situations that remind us we can be broken.

2. When you know, you know.

Mario Gabriele of the Generalist had a great "50 Things" style post a few months ago that I found myself nodding along to almost the whole way through. The glaring exception was Number 33: "Love is easy. The difference between bad relationships and meeting your spouse is not a matter of degree, but category. Compatibility is unignorable and effortless."

I believed this completely, once. I've since come to believe that what's unignorable and effortless is chemistry. Chemistry can be powerful enough to turn your life sideways, but it's not the same thing as compatibility. Compatibility is litigated on the timescale of years and decades; it's not a momentary feeling but a description of what happens during an extended course of change. Do you grow toward one another, or do you grow apart? Does the relationship help you move closer to the person you aspire to be?

I "just knew" with my first husband. I wasn't sure with my second. Four years in, the signs have flipped.

3. If it were a good idea, someone would be doing it already.

There's a belief, especially common among wonky types, that you don't find $20 bills on the sidewalk — if an opportunity were real, someone would have seized it already. This sounds worldly and wise, but it's actually a form of defensive thinking. It assumes the current way is best, that everything worth doing is being done.

In my experience, this is just false. When I started playing poker, I discovered that physical tells were basically a wide-open field — despite the fact that most pros believed the topic was played out. If you think you see an opportunity others have missed, you should have a story for why they're missing it, and you should ask around to stress-test that story. But if no one can give you a satisfying reason, don't assume one exists. Sometimes the answer really is just inertia or fear.

4. If it's worth doing, it's worth doing well.

Almost nothing is worth doing "well." Many things can and should be done to a minimal standard of quality, because anything more is a waste of effort that can be better allocated. The exceptions aren't things that should be done "well," but things that should be done to an exceptionally high standard, probably a much higher one than you see modeled by the people around you.

This is not semantics; most people really "do everything the way they do anything" — whether that's in a low-effort or high-effort way. But the point of phoning it in on a lot of stuff is to save your energy to expend on the 1 in 1000 things that really matter … and then to actually spend it.

A recent example: I decided that I want to narrate my own audiobook for *You Can Just Do Things*. When I told a friend in the industry this and asked whether they knew any good coaches to work with, they said that hiring a coach wasn't normal or necessary, because my publisher would decide based on a sample whether I was good enough at reading to do it, and if so give me a producer to work with day-of. But it "1 in 1000" matters to me whether the audiobook is so good no one would ever think of returning it on account of the narration, so I'm not interested in the normal-shaped solution that constitutes doing it "well," I'm interested in the one that causes me to come to mind when someone asks this question. In this case, that means hiring multiple coaches and practicing for months.

5. If you like your job, you're doing something wrong.

Back when I was more alienated from my emotions, I thought about work like this: You have a budget of energy to spend, and the goal is to efficiently convert that energy into impact. You identify the activity that generates the highest impact per unit of energy, and then you spam that button for as many waking hours as you can stomach. Is it any wonder I was always burning out?

What I failed to understand is that energy is not fixed. It's dramatically affected by what you choose to do — some activities increase your energy over time, others deplete it. This feels so obvious to me now that I feel kind of stupid including it here, but I know many people (they are endemic to the Bay Area) who still labor under the false belief. Whether you enjoy what you're doing affects your ability to do it year in and year out, and dismissing this as "selfish" doesn't make it untrue. As my husband puts it, the best use of effortful motivation is to engineer your life to require less of it.

6. "All people are insane. They will do anything at any time, and God help anybody who looks for reasons."

This line, from Kurt Vonnegut, used to neatly encapsulate the way I understood other people — which is to say, not at all. Trying to figure out why people did what they did, or imagine what they'd do next, seemed like a fool's errand. I was judgmental about it, too: I assumed this was a problem, not with me, but with everyone else.

I didn't fundamentally move beyond this belief until recent years, when I came into contact with the Enneagram. I imagine there are other personality typing systems that can do something similar, but the Enneagram was magical for me because it clued me into the fact that people can have very different motivational systems from my own that are still internally coherent and produce good things in the world.

People's behavior was previously incomprehensible to me because I was imputing my own motivational system to them, trying to figure out what would cause me to act the way they did — essentially, concluding that *their* actions made no sense in light of *my* goals. But people have wildly diverse emotional hardware, which determines what's rational for them. And my particular personality — independent, moralistic, systematic, challenge-seeking, competitive — is unusual, so I was especially poorly placed to measure others by the yardstick of myself.

This shift in perspective was dramatically good for me and for my relationships with other people. It let me see the ecosystem of psychologies as a kind of miracle — wow, it really does take all kinds — rather than a prompt for cosmic horror. And it gave me confidence that, if I

invested in understanding people more deeply, they would eventually cease to be total mysteries, prone to bizarre and unreasonable behavior. As far as I can tell, this is the better part of emotional intelligence.

7. Money can't buy happiness.

The good version of this advice is: You'll still have to work at being happy, even if you're rich. Money in your bank account won't do it for you, and wealth comes with its own pathologies, like endless status-chasing. But the common gloss — that money stops mattering past some modest threshold — is basically false.

You may have heard there's research showing money doesn't buy happiness. That research was explosively popular precisely because it was counterintuitive — it betrayed the commonsense intuition that more money means more freedom, more options, more ability to help people you care about. Turns out it was also riddled with methodological problems. More recent work by Matthew Killingsworth (nominative determinism FTW), using large-scale studies that pinged participants about their happiness at random moments, found steady returns to well-being with rising income, no plateau in sight. The myth persists because it serves a psychological function: It helps people without money feel better about their situation, and helps the wealthy downplay their advantages. But it's not true.

8. Intuition is fake and rationality solves everything.

I used to be horribly judgmental of people who made decisions based on intuition, as in, "I just do what feels right." And then I noticed that whenever I overrode my intuitions, I made *terrible* decisions. I hired the wrong people, started projects with the wrong people, dated the wrong people, ate food that made me feel bad, did forms of exercise I found unfun and injurious. I don't have a satisfying theory for why this is, but I know what happens: When I shift from emotional intuition to a logical story about a decision, it's much easier for my reasoning to get hijacked by plausible-sounding directives that turn out to be nonsense.

It took me a long time to learn this, and that might have something to do with my training in law and poker. Overriding intuition in favor of explicit reasoning makes sense in contexts that reward systematic rationality, where you will get separated from your money if you count on gut feelings. But most of life isn't a closed system with enumerated rules and clear outcomes. If you try to explicitly name all the factors that make someone a good friend or co-founder, you might come up with some good indicators, but your crude map will also mislead you about the territory.

This is why “wisdom is knowledge that can’t be transmitted.” Wise people have navigational skill, not a long list of rules.

Sign up to be notified when my book, You Can Just Do Things, is available for purchase.

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · SOURCE

Look at the past history of this blog. There are many blog posts abou

So mine is a trick. People feel more and more programming is complete

This is why yesterday, on X, I said that I believe many programmers a

1. You can now generate a lot of code, even *not* accounting for the

2. LLMs are very good at writing locally optimal code, and are worse

3. The working day is 8 hours. If you read the code, it is a tradeoff

Controlling the ideas. Do you remember this phrasing from the Mythica

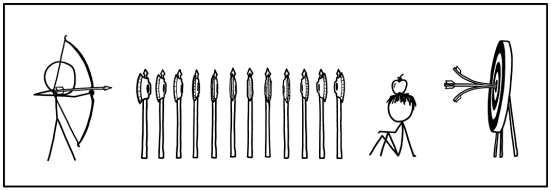
Matteo Collina yesterday asked me, in reply to my tweet: but didn’t y

Fable and GPT 5.6 reviews to the sorted sets memory saving are going

:

Archery Feat

XKCD.COM · 21 AUG 2026 · SOURCE



I WANT TO PROVE THAT MY NEW PROTAGONIST IS A BETTER SHOT THAN ALL THOSE OTHER PROTAGONISTS.

Update: I would not have embarked on this powerscaling venture if I'd known how thoroughly Mark Twain was going to roast me.

Computer History

CODEX CHANGELOG · 13 AUG 2026 · SOURCE

Computer History [Computer History](/codex/customization/computer-history) is an opt-in feature in the ChatGPT desktop app on macOS that turns activity across apps and websites into memories and a timeline that ChatGPT and Codex can use. Choose which apps and websites contribute, pause collection, and review or delete your history at any time. Computer History is available to ChatGPT Pro, Business, and Enterprise users. Business and Enterprise administrators must enable access before workspace members can turn it on. Initial availability excludes the European Economic Area (EEA), Switzerland, and the United Kingdom.